BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

LESSON 021 · Graphs: directed and undirected

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. Which structure is LIFO?

Q2. Which structure is FIFO?

Q3. In an array-based linked list, what does the next pointer store?

Q4. In an undirected graph, if A is connected to B, is B also connected to A?

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

Recall the term — graph

Recall the term — vertex

Recall the term — edge

Recall the term — directed

Recall the term — undirected

Recall the term — adjacency matrix

Recall the term — neighbour

New learning

Create — CODE:
int[,] graph = new int[3,3]; — Three vertices (A, B and C) produce a 3 by 3 matrix: one row and one column per vertex. The four-vertex 4×4 example comes later in You Do.

Add undirected edge — CODE:
graph[a,b] = 1;
graph[b,a] = 1; — The matrix is symmetrical for an undirected connection.

Add directed edge — CODE:
graph[a,b] = 1; — Only A→B is stored.

Remove and inspect — CODE:
graph[a,b] = 0;

for (int v = 0; v < graph.GetLength(1); v++)
{
if (graph[a,v] == 1)
Console.WriteLine(v);
} — Clearing removes an edge; scanning a row finds neighbours.

Worked example

Build A—B—C

1. Make matrix — CODE:
int[,] graph = new int[3,3];

2. A—B — CODE:
graph[0,1] = 1;
graph[1,0] = 1;

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

3. B—C — CODE:
graph[1,2] = 1;
graph[2,1] = 1;

4. Read neighbours of B — CODE:
row 1 = 1 0 1
B connects to A and C

We Do · Practise with guidance

Add undirected edge

Add edge 0—2.

Fill the same code

```
graph[0,2] = ?;
graph[2,0] = ?;
```

```
graph[0,2] = 1;
graph[2,0] = 1;
```

Add directed edge

Add only 1→3.

```
graph[1,3] = ?;
```

```
graph[1,3] = 1;
```

Remove undirected edge

Remove 0—2.

```
graph[0,2] = ?;
graph[2,0] = ?;
```

```
graph[0,2] = 0;
graph[2,0] = 0;
```

Find neighbours

Inspect row 1.

```
if (graph[1,v] == ?)
    Console.WriteLine(v);
```

```
if (graph[1,v] == 1)
    Console.WriteLine(v);
```

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Create an undirected graph
Create a 4×4 matrix and add 0—1, 0—2 and 2—3.

Question 2

List neighbours
Output every vertex directly connected to vertex 0.

Question 3

Modify the graph
Add 1—3, then remove 0—2.

Question 4

Show direction matters
Create a 2×2 directed matrix with 0→1 only, then output both directional entries.

Exit Ticket · Answer before checking

An undirected edge normally changes two entries in an adjacency matrix.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. A stack.

Q2. A queue.

Q3. The array index of another item.

Q4. Yes.

You Do pages · Independent questions

Q1. For an undirected graph, every edge is recorded in both directions.
CODE:
int[,] graph = new int[4,4];
graph[0,1] = graph[1,0] = 1;
graph[0,2] = graph[2,0] = 1;
graph[2,3] = graph[3,2] = 1;
Matrix rows (0–3): 0110, 1000, 1001, 0010. Check that it is symmetrical across the diagonal.

Q2. Inspect row 0: a 1 marks a direct neighbour.
CODE:
for (int v = 0; v < 4; v++)
{
if (graph[0,v] == 1)
Console.WriteLine(v);
}
Output: 1 then 2. Vertex 3 is connected indirectly, not directly.

Q3. Adding an undirected edge sets two cells; removing one clears two.
CODE:
graph[1,3] = graph[3,1] = 1;
graph[0,2] = graph[2,0] = 0;
Edges afterwards: 0—1, 1—3, 2—3.

Q4. A directed edge is written in one matrix cell only.
CODE:
int[,] graph = new int[2,2];

```
graph[0,1] = 1;
Console.WriteLine(graph[0,1]); // 1
Console.WriteLine(graph[1,0]); // 0
```

0→1 exists; 1→0 does not. The matrix need not be symmetrical.

Exit Ticket · Model answer

True. For an ordinary undirected edge between different vertices, the adjacency matrix records both directions.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed: